

From Requirement to API

How to think and work like a Back-End Developer inside a real team



Applied scenario: Job Application Management System

BD

Bahaa Dagher

Backend Developer & Trainer

Who's Training You Today

2+

Years as a Backend Developer

Building real-world APIs and services with .NET

3+

Years in Training & Teaching

Helping developers go from writing code to thinking like engineers

Let's build that same real-world thinking together today.

Today's Agenda

1

Opening — why aren't we opening Visual Studio yet?

2

BRD vs PRD

3

Live Demo: Requirement → PRD

4

Business Rules

5

Data Modeling

6

API Design

7

Acceptance Criteria

8

Full example review

9

Task + Day 2 preview

**If someone came to you right now and said:
" I want an API that lets people apply for a job "**

What exactly would you start doing?

Controller • DbContext • Entity • Migration • Repository

All of this matters — but it's not the first thing you should do.

From Developer to Engineer

Developer

Knows how to write working code and use the tools correctly

Engineer

Knows how to turn a vague business need into a clear, testable technical solution

↑ *that's what these 6 days will build*

The thinking journey before any line of code



The questions that get you to Understand & Define:

- Who is the user?
- What are the business rules?
- What data is needed?
- What do they want to do?
- When is the request valid?
- When is the feature considered done?

BRD × PRD

The difference between business language and product language

BRD

Business Requirements
Document

Answers: Why does the business need this?

Business Problem

The hiring company receives job applications through different channels, making it hard to track candidates.

Business Goal

A system that lets candidates apply for jobs and track their application status.

A BRD speaks the language of the business — no technical detail.

PRD

Product Requirements
Document

Answers: What exactly should the system do?

Feature	Apply for Job
Actor	Candidate
Requirement	The candidate can apply for an available job
Input	CandidateId, JobId, CV
Rules	Job must be active — Candidate cannot apply twice
Output	Application Created

BRD vs PRD

	BRD	PRD
Language	Business language	Product / technical language
Question	Why do we need this?	How exactly will it work?
Content	Problem + Goal	Feature + Rules + Data + API
Audience	Management	The technical team

PRD Components

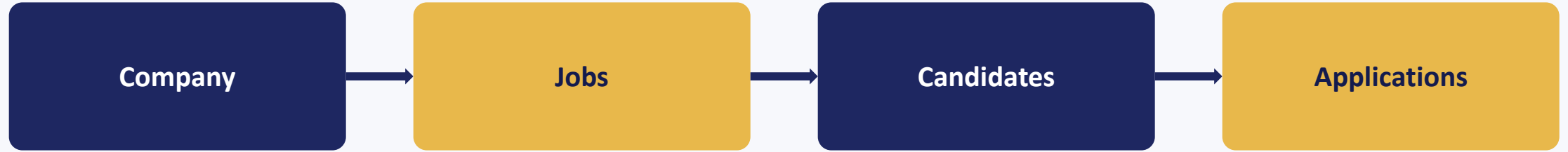
Before we dive into any details — here's the full shape a PRD should reach

1	Business Problem	Why the feature is needed in the first place
2	Feature	The name and description of the feature
3	Actors	Who will use this feature
4	Business Rules	The rules that govern the feature's behavior
5	Data Model	The entities, their fields, and relationships
6	API Endpoints	The endpoints that implement the feature
7	Acceptance Criteria	When we can say the feature is truly done

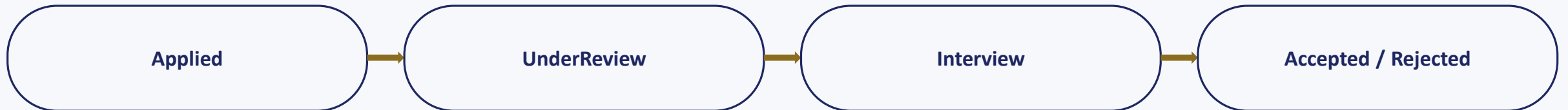
Job Application Management System

The scenario we'll build on through the Capstone

The scenario, top to bottom



And the application status moves through:



A candidate can apply for a job, and a recruiter can review applicants and update the application status.

Live Requirement

"Candidate should be able to apply for a job. "

We'll turn it into a full PRD together, step by step

The questions that clarify the requirement

1

Who can apply?

Candidate only, and must be registered

2

Can any job receive applications?

No, it must be active

3

Can the same person apply twice?

No

4

What happens after applying?

Status = Applied

5

What data is needed?

CandidateId, JobId, AppliedAt, Status, CVUrl

Result: PRD — Apply for Job

Apply for Job	Feature
Candidate	Actor
CandidateId, JobId, CV	Input
Job Active — No duplicate application	Rules
Application Created (Status = Applied)	Output

Business Rules

You're not just Data Entry — you're the one enforcing the rules

Business Rules — Apply for Job

Only registered candidates can apply.

BR-01

Candidate can apply only to active jobs.

BR-02

Candidate cannot apply to the same job more than once.

BR-03

Every new application starts with the "Applied" status.

BR-04

Discussion question

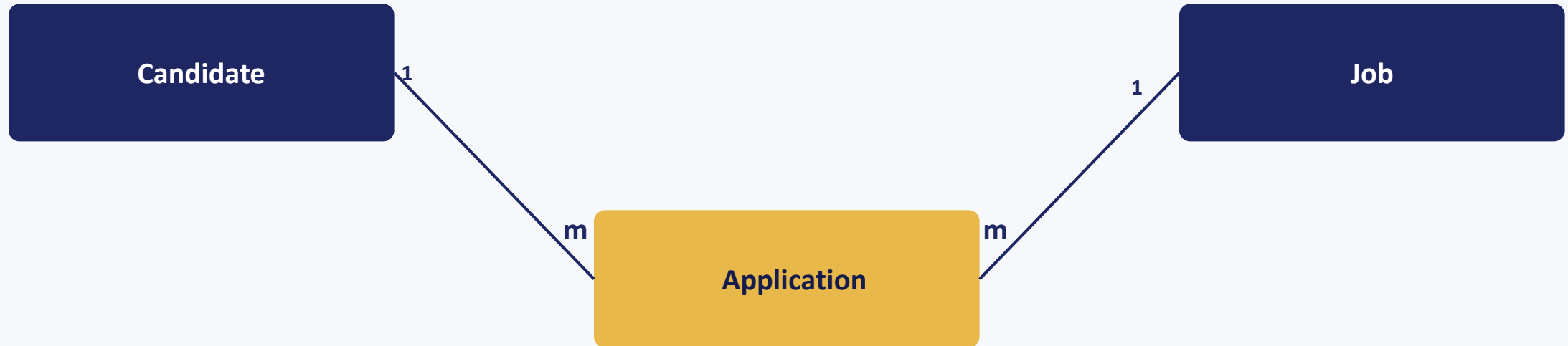
Which of these business rules should also be enforced on the front-end?

Even if the front-end validates, the back-end must enforce the rule again — there's no guarantee the request came from that same UI.

Data Model

From Business Rules to Entities

Relationships between entities



Is Application just a link table between entities?

No — it has its own data (Status, AppliedAt, CVUrl), so it's a standalone entity.

Entity fields

Candidate

- Id
- Name
- Email
- CVUrl

Job

- Id
- Title
- Description
- IsActive

Application

- Id
- CandidateId
- JobId
- Status
- AppliedAt

API Design(endpoints)

From Data Model to Endpoints

Resource, not Action

✓ `POST /api/applications`

✗ `POST /api/applyForJob`

✓ `GET /api/jobs`

✗ `GET /api/getAllJobs`

The URL represents the Resource (Application / Job) — not the Action.

Required endpoints

GET	/api/jobs	List available jobs
GET	/api/jobs/{id}	Get job details
POST	/api/applications	Apply for a job
GET	/api/applications/my	Candidate's own applications
GET	/api/jobs/{jobId}/applications	Applicants for a job (Recruiter)

Acceptance Criteria

When does the developer know the feature is done?

✗ Not: "it works on my machine"

✓ But: when the Acceptance Criteria are met

Given / When / Then

Scenario 1 — Apply for active job

- Given** an active job and a valid candidate
- When** the candidate submits an application
- Then** the application is created with status "Applied"

Scenario 2 — Duplicate application

- Given** the candidate already applied
- When** they submit another application
- Then** the API rejects the request

Another Example — Review Application & Update Status

Business Problem:

Feature:

Actor:

Business Rules

-

Data Model (delta)

-

API Endpoints

- GET

 /api/
- GET

 /api/
- PUT

 /api/

Acceptance Criteria

-

The full journey



This is what every trainee should now be able to do on their own

Your Turn — The Exercise

Activity Feature: Cancel Application

1 Business Problem (3-5 lines)

3 Actor / Actors

5 Data Entities

7 Acceptance Criteria

2 One Feature

4 Business Rules

6 API Endpoints

Team discussion → Individual submission

Before you submit — review yourself

- ☐ Did you understand the problem?
- ☐ Are the Rules testable?
- ☐ Are the API Endpoints clear?
- ☐ Did you identify the Actor?
- ☐ Are the Entities logical?
- ☐ Are the Acceptance Criteria testable?

Exercise Solution — Cancel Application

Business Problem: Some candidates apply by mistake or change their mind, with no formal way to withdraw — leaving the application pending and wasting the recruiter's time.

Feature: Cancel Application **Actor:** Candidate

Business Rules

- Candidate can only cancel their own applications
- Cancelling is allowed only while status is Applied or UnderReview
- Once status reaches Interview or beyond, cancelling is not allowed
- A cancelled application gets status Cancelled — it isn't deleted
- A cancelled application can't be reopened; a new application must be submitted

Data Model (delta)

- Application: Status → add “Cancelled”, + CancelledAt (nullable)

API Endpoints

DELETE /api/applications/{id}

Acceptance Criteria

- Status "Applied", owned by requester → status becomes "Cancelled", CancelledAt recorded
- Status "Interview" or beyond → API rejects the request
- Application doesn't belong to the requester → 403 Forbidden
- Application already "Cancelled" → API rejects the request

Take-Home Task

For extra practice — same PRD structure, one more feature

Task Feature: Close Job

- Write the same 7-part PRD you practiced today for this feature.
- Actor: Recruiter — closing a job stops it from receiving new applications.
- Think about what should happen to applications that already exist.

Coming up — Day 2:

PRD → Delivery Plan — Breaking requirements into epics, stories & tasks